



Universidad de Margarita
Vicerrectorado académico
Decanato de Ingeniería y afines
Sistemas III
Prof.: Ing. Eyla Vásquez

Evaluación – Proyecto: Avance III

1. Propósito de la evaluación

El Avance III es la entrega final del proyecto integrador. El equipo presenta el sistema funcionando, con su documentación técnica completa y defiende oralmente las decisiones tomadas durante el desarrollo. No es solo entregar código, es demostrar que el equipo comprende lo que construyó y puede justificarlo técnicamente, como ocurre en el mundo real.

La defensa oral dura 15 minutos por equipo y se organiza como una revisión técnica: el equipo muestra el sistema corriendo, navega por el código y responde preguntas directas sobre las decisiones que tomó. No hay presentación preparada ni diapositivas, solo el sistema, el código y las respuestas del equipo.

1.1 Competencias evaluadas

- Entregar un sistema funcional que corre en Docker con las funcionalidades principales operativas.
- Demostrar la aplicación de principios SOLID, herencia vs. composición y al menos un patrón de diseño en el código real.
- Presentar y explicar las pruebas unitarias escritas: qué prueban, por qué existen y qué pasaría si faltaran.
- Mostrar el Dockerfile y el docker-compose.yml y explicar qué hace cada instrucción.
- Presentar el README del proyecto y demostrar que permite a alguien externo instalar y usar el sistema.

- Mostrar la documentación de la API (Swagger/OpenAPI o tabla de endpoints) y verificar que coincide con el sistema real.
- Defender las decisiones técnicas tomadas durante el desarrollo: por qué se organizó el código así, qué se cambiaría si se hiciera de nuevo.

2. ¿En qué consiste el Avance II?

El Avance III tiene dos partes: un entregable escrito y una defensa oral de 15 minutos.

2.1 Entregable escrito

El equipo entrega el repositorio completo del proyecto con todo lo siguiente incluido:

Entregable	Descripción	Dónde debe estar
Código fuente	El sistema completo funcionando, organizado según la arquitectura descrita en el Avance II.	Repositorio (GitHub, GitLab o similar)
README.md	Documentación de instalación, uso, endpoints principales y cómo correr las pruebas. Debe permitir a alguien externo instalar el sistema desde cero.	Raíz del repositorio
Dockerfile	Archivo que empaqueta la aplicación en un contenedor. Debe funcionar con docker build.	Raíz del repositorio o carpeta del servicio
docker-compose.yml	Archivo que levanta el sistema completo (app + base de datos si aplica) con docker compose up.	Raíz del repositorio
Pruebas unitarias	Mínimo 3 pruebas con asserts significativos	Carpeta de pruebas del proyecto

Entregable	Descripción	Dónde debe estar
Documentación de la API	Swagger UI (/docs en FastAPI), tabla de endpoints en el README, o archivo OpenAPI.	Accesible con el sistema corriendo o en el README

2.2 Defensa oral — 15 minutos por equipo

La defensa no es una presentación preparada. Es una revisión técnica en vivo: el equipo tiene el sistema corriendo en su computadora y la profesora hace preguntas directas sobre el código, las pruebas, el Docker y las decisiones tomadas. Los 15 minutos se distribuyen así:

Momento	Duración	Qué ocurre
Sistema funcionando	3 min	El equipo muestra el sistema corriendo — ya sea con docker compose up o localmente. Se navega por al menos una funcionalidad principal completa.
README y documentación	3 min	El equipo abre el README y demuestra que contiene lo necesario para que alguien externo instale y use el sistema. Se verifica que la API documentada coincide con lo que el sistema realmente hace.
Pruebas unitarias	3 min	El equipo abre el archivo de pruebas, explica qué está probando cada test y por qué existe. Se ejecutan las pruebas en vivo con el comando correspondiente y workflow correspondiente
Dockerfile y Docker Compose	3 min	El equipo abre los archivos y explica qué hace cada instrucción relevante. Se verifica que docker compose up levanta el sistema correctamente.
Preguntas técnicas	3 min	La profesora pregunta sobre decisiones de diseño: SOLID aplicado, patrón de diseño usado, organización del código, algo que cambiarían si lo hicieran de nuevo.

3. Requisitos del sistema entregado

3.1 Funcionalidad mínima

El sistema debe implementar al menos las funcionalidades descritas en las User Stories del Avance I. **Un sistema que no corre no puede defenderse y si un integrante de equipo no está, no tiene nota.**

3.2 Dockerfile y Docker Compose

El sistema debe poder levantarse con Docker. El equipo debe poder explicar cada instrucción de los archivos durante la defensa.

El Dockerfile debe:

- Partir de una imagen base oficial (python:3.x, php:8.x, node:x, etc.).
- Instalar las dependencias del proyecto dentro del contenedor.
- Copiar el código al contenedor.
- Exponer el puerto correcto.
- Definir el comando que arranca la aplicación.

El docker-compose.yml debe:

- Definir al menos el servicio de la aplicación.
- Si el sistema usa base de datos, incluirla como servicio separado.
- Mapear los puertos correctamente para que el sistema sea accesible desde el navegador o Postman.
- El comando docker compose up --build debe levantar el sistema sin pasos adicionales manuales.

3.3 Pruebas unitarias

El equipo debe tener al menos 3 pruebas unitarias escritas con el framework de pruebas correspondiente a su lenguaje (pytest, PHPUnit, Jest, JUnit, etc.).

3.4 README profesional

El README debe permitir que alguien que nunca vio el proyecto pueda instalarlo y usarlo sin preguntar nada. Debe contener:

- Nombre del proyecto y descripción en 2-3 oraciones de qué hace.
- Requisitos previos: qué debe tener instalado (Docker, Python, Node, etc.).
- Instrucciones de instalación paso a paso con los comandos exactos.
- Cómo levantar el sistema con Docker (docker compose up).
- Lista de los endpoints principales o cómo acceder a la documentación de la API.
- Cómo ejecutar las pruebas unitarias.

3.5 Documentación de la API

Si el sistema tiene endpoints que devuelven JSON, deben estar documentados. La forma de documentación depende del framework:

Framework	Documentación esperada
FastAPI (Python)	Swagger UI disponible en /docs con el sistema corriendo. Debe mostrar todos los endpoints.
Django REST	DRFBrowsable API o swagger con drf-spectacular. Alternativamente, tabla completa en el README.
Laravel (PHP)	Tabla de endpoints en el README o documentación con L5-Swagger.
Express / Node.js	Tabla de endpoints en el README con verbo, ruta, descripción, request y response.
Spring Boot	Swagger UI con springdoc-openapi o tabla completa en el README.
Cualquier framework	Como mínimo: tabla en el README con verbo HTTP, ruta /api/v1/..., qué recibe y qué devuelve.

4. Guía de la defensa oral

La defensa no se prepara con diapositivas. Se prepara teniendo el sistema funcionando y el código abierto. El equipo debe estar en capacidad de navegar por cualquier archivo del proyecto y explicar lo que hace.

4.1 Cómo prepararse para la defensa

- Asegurarse de que docker compose up --build levanta el sistema sin errores el día antes de la defensa.
- Tener VS Code (o el editor del equipo) abierto con el proyecto cargado.
- Saber ejecutar las pruebas con un solo comando y que todas pasen.
- Poder navegar al Dockerfile, docker-compose.yml, archivo de pruebas y README en menos de 30 segundos.
- Cada integrante debe poder responder preguntas sobre cualquier parte del proyecto, no solo la parte que escribió.

6. Entrega del informe

6.1 Conclusión del proyecto

Esta sección cierra el informe, no es un resumen de lo ya escrito, es una **reflexión crítica del equipo** sobre su propio proceso de aprendizaje y sobre el cumplimiento real de los objetivos planteados desde el Avance I.

El equipo debe responder, con sus propias palabras, los siguientes puntos:

a) Cumplimiento de objetivos

Retomar los objetivos definidos en el Avance I (sección 3.2) y, uno por uno, explicar cómo fueron alcanzados en el sistema final. Si algún objetivo no se cumplió completamente, decirlo con honestidad y justificar por qué (alcance reducido, limitación técnica, decisión de equipo, tiempo, etc.)

b) Aplicación de los temas de la asignatura

Explicar cómo se aplicaron en el proyecto real los conceptos vistos en clase:

- Principios SOLID: cuáles se aplicaron y dónde, con ejemplos concretos del código.
- Herencia vs. composición: qué se usó y por qué se eligió esa opción sobre la otra.
- Patrón(es) de diseño: cómo resolvió un problema real del proyecto (no solo mencionarlo, sino explicar la diferencia que hizo tenerlo).

- Modelado UML, arquitectura por capas, pruebas unitarias, Docker: qué aprendieron al llevar estos conceptos de la teoría a un sistema que realmente corre.

c) Aporte del proyecto al aprendizaje del equipo

Reflexionar sobre qué le dejó el proyecto al equipo en general: qué fue lo más difícil, qué entendieron mejor al tener que defenderlo oralmente, qué harían diferente si empezaran de nuevo con lo que saben ahora.

d) Cierre

Un párrafo final breve que sintetice, en su propia voz, si el sistema construido resuelve el problema planteado en el Avance I y por qué.

NOTA: La conclusión debe ser una redacción formal redactada en tercera persona, además de usar conectores en relaciones un punto con el otro ya que la redacción debe ser continua

6. Formato y entrega

- Trabajar en el mismo documento en Google Docs del avance I y conservar mis permisos de comentador a mi correo evasquez.1881@unimar.edu.ve
- Invitar al correo evasquez.1881@unimar.edu.ve con el rol de lectura en el repositorio GitHub el cual deben hacer entrega en moodle
- Usar la normativa de la universidad para la realización del trabajo, portada, márgenes, tipo de letra, etc